

第三章：内存管理

一、核心概念

1.1 虚拟内存 (Virtual Memory)

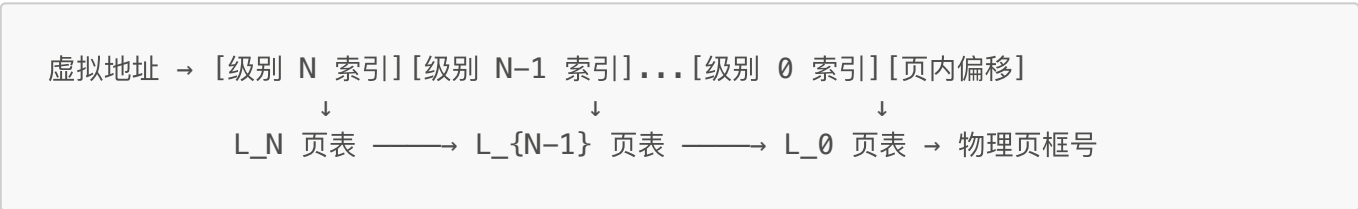
虚拟内存是操作系统最重要的抽象之一。每个进程拥有独立的虚拟地址空间，通过页表映射到物理内存。

三大好处：

- **隔离**：进程 A 无法访问进程 B 的内存
- **抽象**：每个进程都"以为"自己独占整个地址空间
- **超分**：虚拟地址空间可以远大于物理内存（按需分配）

1.2 页表 (Page Table)

页表是虚拟地址到物理地址的映射表。现代系统使用多级页表以节省空间：



架构	页表级数	虚拟地址宽度	页大小
RISC-V Sv39	3 级	39 位 (512GB)	4KB
RISC-V Sv48	4 级	48 位 (256TB)	4KB
x86-64	4 级 (PML4)	48 位	4KB
ARM64	4 级	48 位	4KB/16KB/64KB

PTE (Page Table Entry) 中的标志位：

- **V (Valid)**：该映射是否有效
- **R/W/X**：读/写/执行权限
- **U**：用户态是否可访问
- **A/D (Accessed/Dirty)**：硬件自动设置，供操作系统统计使用
- **RSW**：软件保留位，可自定义用途（如 COW 标记）

1.3 物理内存分配

操作系统需要管理空闲的物理页框。两种经典分配器：

伙伴系统 (Buddy Allocator)：

- 管理 2^n 大小的连续页块
- 分裂：大块拆成两个等大的"伙伴"
- 合并：释放时检查伙伴是否也空闲，若是则合并成更大块

- 优点：能快速分配连续物理页，碎片可合并
- 缺点：只能分配 2^n 大小，存在内部碎片

Slab 分配器：

- 在伙伴系统之上，为固定大小的小对象提供高效分配
- 预先将整页切成等大的对象槽（slot），用 bitmap 管理已分配/空闲
- 优点：小对象分配接近 $O(1)$ ，减少内部碎片
- 变体：Linux SLAB → SLUB → SLOB

1.4 按需分页（Demand Paging）

惰性映射：`mmap()` 或 `malloc()` 时只记录映射关系（VMA），不分配物理页。当进程实际访问该地址时，触发 Page Fault，内核在 fault handler 中才分配物理页并建立映射。

好处：大幅减少物理内存消耗。`malloc(1GB)` 可能只消耗几 KB 的页表空间。

1.5 写时复制（Copy-on-Write, COW）

fork 时不立即复制父进程的所有物理页，而是让父子共享同一物理页，并将页表项标记为只读。当任一方尝试写入时：

1. 触发 Store Page Fault
2. 内核检查 COW 标记
3. 复制该页，建立独立映射
4. 修改 PTE 为可写

引用计数：每个物理页维护引用计数。COW 复制时 `refcount--`；当 `refcount == 1` 时直接修改权限，无需复制。

1.6 VMA（Virtual Memory Area）

VMA 是虚拟地址空间中一段连续区域的描述符，记录该区域的"语义"：

```
struct vma {
    start, end    // 虚拟地址范围
    prot          // 权限（PROT_READ | PROT_WRITE | PROT_EXEC）
    flags         // MAP_PRIVATE | MAP_SHARED | MAP_ANONYMOUS
    file          // 关联文件（匿名映射为 NULL）
    offset        // 文件偏移
};
```

VMA vs PTE：VMA 记录"应该是什么"（语义），PTE 记录"当前实际映射了什么"（状态）。两者可以不一致——这就是惰性映射的基础。

1.7 零页优化

全局共享一个全零的物理页 (`zero_page`)。首次读访问映射到 `zero_page`（只读共享），首次写访问才分配新页。这是 COW 思想在匿名映射上的应用。

二、基本推论

推论 1：多级页表是空间和时间的权衡。单级页表需要 2^{27} 个条目（Sv39），占 1GB 内存。三级页表让大部分未使用的虚拟地址区域不需要分配页表页，但每次地址翻译需要 3 次内存访问（被 TLB 缓解）。

推论 2：Page Fault 是惰性机制的统一接口。无论是惰性映射、COW、还是 mmap 文件映射，都通过同一个 Page Fault handler 入口处理，根据 VMA 信息决定具体动作。

推论 3：buddy 和 slab 是互补的。buddy 擅长大块连续分配（DMA 缓冲区），slab 擅长小对象分配（进程描述符、inode 等）。两者组合覆盖了内核的全部内存需求。

推论 4：COW 让 fork 接近零开销。fork 只复制页表项并设置 COW 标记， $O(\text{页表项数})$ 。如果子进程立即 exec，那么 fork 复制的页表项会被全部丢弃，真正的物理页拷贝为零。

推论 5：VMA 的 split/merge 是 munmap/mprotect 复杂度的根源。munmap 一个 VMA 中间部分需要将一个 VMA 拆成两个；mprotect 修改部分区域的权限也需要先 split。这些操作必须保证原子性——不能出现半成品状态。

三、Linux / 通用实现

3.1 Linux 的物理内存管理

Linux 使用伙伴系统管理物理页框，分为多个 zone：

ZONE_DMA	(0-16MB)	— ISA DMA 设备
ZONE_DMA32	(0-4GB)	— 32位 DMA 设备
ZONE_NORMAL	(4GB+)	— 普通内存
ZONE_HIGHMEM	(仅32位系统)	— 需要临时映射的高端内存

每个 zone 维护 11 个 order 的 free_area 链表（order 0 = 4KB，order 10 = 4MB）。

SLUB 分配器（Linux 默认）：

- 每个 CPU 有私有的 partial slab 列表，减少锁竞争
- 按对象大小分组（kmalloc-8, kmalloc-16, ..., kmalloc-8192）
- 还有专用缓存（`kmem_cache_create`）用于高频对象（task_struct, inode 等）

3.2 Linux 的 VMA 管理

Linux 5.x 之前用红黑树管理 VMA，6.1+ 改用 **maple tree**：

```
struct vm_area_struct {
    unsigned long vm_start, vm_end;
    struct mm_struct *vm_mm;
    pgprot_t vm_page_prot;
    unsigned long vm_flags;
    struct file *vm_file;
    unsigned long vm_pgoff;
```

```
const struct vm_operations_struct *vm_ops;
};
```

Page Fault 处理 (`handle_mm_fault`) :

1. 查找 VMA: `find_vma(mm, address)`
2. 权限检查: 写访问 vs VMA 的 `VM_WRITE`
3. 分配页表项: `__handle_mm_fault()` → `handle_pte_fault()`
4. 根据情况:
 - 匿名页首次访问 → `do_anonymous_page()` → 分配零页
 - 文件映射首次访问 → `do_fault()` → 从 page cache 读取
 - COW 写 → `do_wp_page()` → 复制页
 - Swap 换入 → `do_swap_page()`

3.3 Linux 的 mmap 实现

```
sys_mmap() → do_mmap()
1. 查找空闲虚拟地址区间 (arch_get_unmapped_area)
2. 创建 VMA 并插入红黑树/maple tree
3. 如果是 MAP_POPULATE → 立即分配物理页
4. 否则 → 只记录 VMA, 实际分配延迟到 Page Fault
```

3.4 Linux 的 COW 实现

```
fork() → copy_mm() → dup_mmap()
→ 遍历父进程所有 VMA
→ 对每个可写的私有映射:
  - 清除 PTE_WRITE
  - 设置 PTE_RDONLY
  - 物理页 page_count++
→ 子进程获得相同的只读页表
```

写入时:

```
do_wp_page()
→ 检查 page_count
→ == 1 → 直接设为可写 (最后一个引用者)
→ > 1 → 分配新页, 复制内容, 映射新页
```

四、RUOS 的实现

核心文件

- `src/mm/kalloc.c` — buddy + slab 物理分配器
- `src/mm/vm.c` — Sv39 三级页表、内核/用户虚拟内存
- `src/mm/vma.h` / `vma.c` — VMA 惰性映射
- `src/mm/shm.h` — System V 共享内存
- `src/memlayout.h` — 地址布局常量

4.1 buddy + slab 两层分配器

xv6 原版只有单页 freelist，无法分配小对象（浪费 99%），也不能分配连续多页（DMA 需要）。

RUOS 的解决方案：

```

请求
├── <= 2KB → Slab 分配器
│   │   7 个 size class: 32, 64, 128, 256, 512, 1024, 2048
│   │   每个 slab 占一页, bitmap 管理已分配的对象
└── > 2KB → Buddy 分配器
    │   15 个 order: 4KB, 8KB, ..., 128MB
    │   分裂/合并保持 2^n 对齐
  
```

buddy 分裂/合并：

```

分配 8KB (order 1):
  free_list[1] 空? → 从 free_list[2] 拿 16KB
  → 切成两个 8KB buddy → 一个返回, 一个放回 free_list[1]

释放 8KB:
  buddy 地址 = addr ^ (1 << (order + 12))
  buddy 也空闲? → 合并成 16KB → 递归合并
  
```

对比 Linux：Linux 的伙伴系统是 11 阶（4KB-4MB），RUOS 是 15 阶（4KB-128MB）。Linux 的 SLUB 有 per-CPU cache 和 NUMA 感知，RUOS 的 slab 是全局锁保护的简单 bitmap。

优点：实现简洁（~500 行），覆盖了所有内存分配需求。

缺点：全局锁在多核场景下可能成为瓶颈（但 QEMU 2 核环境下不是问题）。

4.2 Sv39 三级页表

```

虚拟地址 (39位):
[38:30] VPN[2] → L2 页表索引 (512 entries)
[29:21] VPN[1] → L1 页表索引
[20:12] VPN[0] → L0 页表索引
[11:0]  offset → 页内偏移 (4KB)
  
```

PTE 位使用：

位	标准用途	RUOS 特殊用途
V	有效	
R/W/X	读/写/执行	
U	用户态可访问	
A/D	访问/脏页	
RSW[8]	软件保留	COW 标记

4.3 COW + 零页

fork 时:

```
uvmcopy(parent_pt, child_pt, sz):
    for each page:
        PTE 设为只读 + COW 标记
        物理页 refcount++
        // 不复制物理页!
```

Store Page Fault 时:

```
cow_alloc(pagetable, va):
    if (refcount == 1):
        直接改权限为可写, 清 COW    // 最后一个引用者
    else:
        new_pa = kalloc()
        memmove(new_pa, old_pa, PGSIZE)
        映射 va → new_pa (可写)
        refcount[old_pa]--
```

零页优化: 全局共享 zero_page, `uvmmalloc_lazy()` 标记为"未分配"。首次读 → 映射到 zero_page (只读), 首次写 → 分配新页。

对比 Linux: 流程一致。Linux 通过 `struct page` 的 `_refcount` 字段做引用计数, RUOS 用单独的 refcount 数组。

4.4 VMA 惰性映射

设计哲学: VMA 记录语义, PTE 记录状态。两者可以不一致。

```
sys_mmap(addr, len, prot, flags, fd, offset):
    va = 找到空闲虚拟地址
    vma_add(proc, va, va+len, prot, flags, file, offset)
    return va // 没有 kalloc, 没有 mappages!

// Page Fault 时才加载:
```

```

vma_handle_fault(va, is_write):
    vma = vma_find(proc, va)
    if (MAP_ANONYMOUS):
        pa = kalloc() + memset(0)
    else:
        pa = kalloc()
        readi(vma->file, ..., offset, PGSIZE) // 从文件读
        mappages(pagetable, va, PGSIZE, pa, prot_to_pte(vma->prot))

```

对比 Linux: Linux 的 `handle_mm_fault()` 逻辑更复杂 (page cache、readahead、swap)，但核心流程相同——查 VMA → 根据类型分配/加载页 → 建立映射。

4.5 VMA split/unmap

munmap 的四种情况:

```

VMA: [=====]

Case 1: 覆盖整个 VMA → 直接删除
      [xxxxxxxxxx]

Case 2: 左侧 → 调整 start
      [xxxx      ]

Case 3: 右侧 → 调整 end
      [      xxxx]

Case 4: 中间 → split 成两个 VMA
      [  xxxx  ] → [===] [===]

```

Case 4 最复杂——需要分配新 VMA 节点，正确计算两半的 offset 和 len。如果分配失败必须回滚。

mprotect 的两阶段方法: 先 split (如果修改范围不对齐 VMA 边界)，再修改权限。避免"改了一半权限然后 split 失败"的不一致状态。

对比 Linux: Linux 的 VMA 用红黑树/maple tree 管理，支持自动合并相邻的同属性 VMA。RUOS 用链表， $O(n)$ 查找，但 VMA 数量少时够快。

4.6 物理/虚拟地址布局

物理地址:

```

0x10000000  UART0
0x10001000  VirtIO disk
0x10002000  VirtIO net
0x0C000000  PLIC
0x80000000  KERNBASE (内核加载基址)
0x80200000  _entry (OpenSBI 跳转地址)
0x88000000  PHYSTOP (128MB)

```

用户虚拟地址：

```
0x00001000  .text / .data
             heap (brk 向高地址增长)
             ...
             mmap 区域
             ...
             stack (向低地址增长)
             [guard page]
TRAPFRAME   trapframe
TRAMPOLINE   trampoline 页 (MAXVA - PGSIZE)
```

4.7 内核栈 1 页→2 页

Bug: `store page fault: stval=0x3ffff8000`，访问了内核栈下方的 guard page。

根因: FAT32 路径解析 + 文件操作嵌套 6-7 层函数，局部变量超过 4KB。

修复: `KSTACK_PAGES=2` (8KB)，与 Linux 的默认内核栈大小一致。

五、八股 / 面试高频问题

Q: 虚拟内存的作用? A: 三个核心: (1) 隔离——进程不能访问彼此的内存; (2) 抽象——每个进程独占完整地址空间; (3) 超分——虚拟空间可远大于物理内存 (按需分配)。

Q: 多级页表为什么比单级页表好? A: 单级页表需要连续的大数组 (Sv39 需要 512K 条目)。多级页表只为实际使用的虚拟地址区域分配页表页，大量未使用的区域不占空间。代价是每次地址翻译需要多次内存访问 (被 TLB 缓解)。

Q: TLB 是什么? 什么时候需要刷新? A: TLB 是页表的硬件缓存。切换进程 (页表基址变化)、修改页表项 (munmap/mprotect/COW) 时需要刷新。多核还需 TLB shutdown (IPI 通知其他核刷新)。

Q: 什么是缺页异常 (Page Fault)? 有哪些类型? A: 访问的虚拟页没有有效映射时触发。三种: (1) 取指缺页——代码段未映射; (2) 读缺页——数据未映射或无读权限; (3) 写缺页——COW 页、惰性映射、无写权限。

Q: 伙伴系统的原理? A: 管理 2^n 大小的连续物理页块。分配时从高阶拆分，释放时与“伙伴”合并成更大块。优点: 能分配连续物理页 (DMA 需要)，碎片可合并。缺点: 只能分配 2^n 大小，有内部碎片。

Q: slab 分配器的作用? A: 在伙伴系统之上，为固定大小的小对象 (几十到几千字节) 提供高效分配。预先将整页切成等大 slot，用 bitmap 管理。避免每个小对象都 kalloc 一整页的浪费。

Q: 什么是 COW (写时复制)? A: fork 时不复制物理页，父子共享并标记只读。写入时触发 Page Fault，内核复制该页。通过引用计数判断是否是最后一个引用者——如果是，直接修改权限无需复制。

Q: mmap 是什么? 惰性映射的好处? A: mmap 将文件或匿名内存映射到虚拟地址空间。惰性映射只记录 VMA (语义)，不分配物理页。实际访问时触发 Page Fault 才分配。好处: malloc(1GB) 只消耗页表开销，直到实际使用才消耗物理内存。

Q: 内核的 OOM Killer 是什么？ A: 当系统物理内存耗尽时，Linux 的 OOM Killer 选择一个进程杀死以释放内存。选择依据是 oom_score（考虑内存使用量、运行时间、优先级等）。

六、项目贡献亮点

- 实现了 buddy+slab 两层物理内存分配器，替代 xv6 的简单 freelist，消除小对象 99% 内部碎片
- 实现了 VMA 惰性映射 + COW + 零页优化，fork 接近零开销
- 实现了 mmap/munmap/mprotect 的完整 VMA split/merge 逻辑
- buddy+slab 架构与 Linux 内核的 buddy+SLUB 一脉相承，面试可直接对照讲解

七、设计取舍总结

决策	RUOS 选择	Linux 做法	权衡
物理分配器	buddy+slab	buddy+SLUB+per-CPU cache	简单但无多核优化
VMA 数据结构	链表	红黑树 / maple tree	O(n) 查找，但 VMA 少时够用
COW	PTE RSW 位 + refcount 数组	struct page._refcount	思路一致，实现更简
零页	全局共享 zero_page	同 (ZERO_PAGE 宏)	完全一致
mmap 策略	惰性映射 + VMA	同 (demand paging)	核心思路一致
共享内存	System V shmget/shmat	System V + POSIX shm_open	只实现 System V
页表	Sv39 三级	x86: 4级, ARM: 4级	RISC-V 标准